ACADEMIC SUBMISSION REPORT: TOXIC TOKEN

Vulnerability Lab Concept: JWT Cracking to Authenticated RCE
Target Machine Architecture: Local Flask Web Deployment Stack
Submission Timestamp: October 2026 | Deadline Verification Trace

1. Executive Summary

This security assessment report evaluates a structured multi-stage web exploitation chain executed within a controlled Python/Flask laboratory environment. The lab objective, designated as **"Toxic Token"**, illustrates the systemic operational risks introduced when weak cryptographic primitives combine with inadequate access control rules and insecure system command processing parameters.

The lifecycle of the compromise follows a linked three-stage solve path. Initially, an offline cryptographic dictionary attack compromises the web platform's active session signature key. By weaponizing this cracked secret, session integrity is systematically bypassed, enabling explicit role privilege vertical escalation. Finally, access controls protecting internal administrative features collapse, allowing dangerous command string interpolation vectors to achieve unconstrained Remote Code Execution (RCE) on the local operating system context, successfully compromising the static data flag payload.

2. Technical Vulnerability Analysis

The operational breakdown of the local web server layer stems from three distinct weaknesses categorized under the Common Weakness Enumeration (CWE) framework:

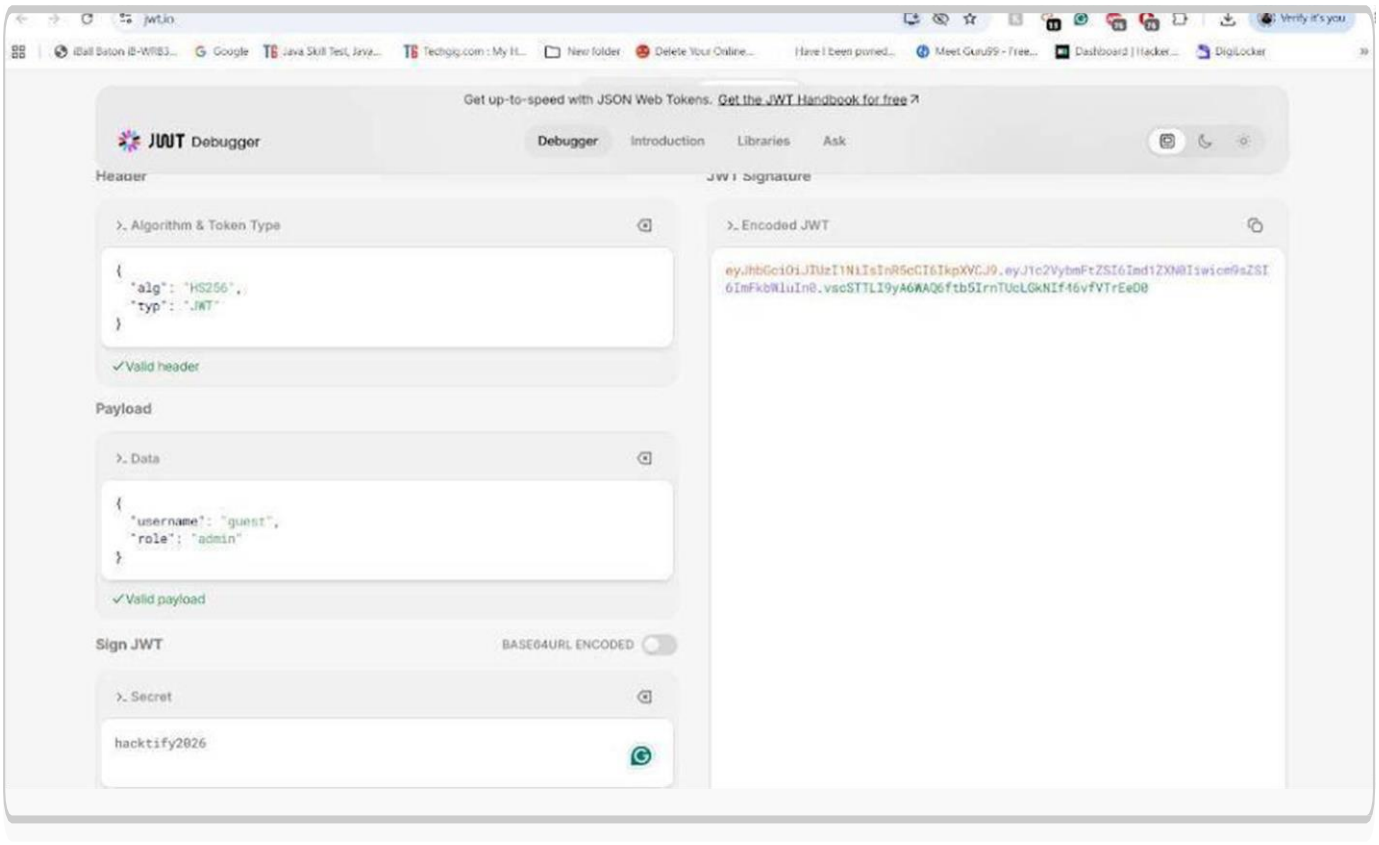
This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

Vulnerability Vector	CWE ID	Architectural Impact & Root Cause Analysis
Weak Cryptographic Key Selection	CWE-522	The platform relies on a static, low-entropy, dictionary-based signing phrase (hacktify2026) under the symmetric HS256 algorithm. This allows an offline adversary to match signatures at scale, stripping the application of cryptographically verifiable token validation integrity.
Broken Role Authorization Controls	CWE-863	The application framework accepts input context structures directly from untrusted client browser data headers without independent server-side verification checkpoints, blindly evaluating modified structural authorization attributes.
OS Command Injection Context Escape	CWE-78	The troubleshooting debug utility passes raw, unfiltered web input data forms into the operating system command handler via raw context mapping execution blocks, enabling parameter chaining escapes.

3. Step-by-Step Proof of Concept (PoC)

Stage 1: Session Interception & Weak Key Recovery

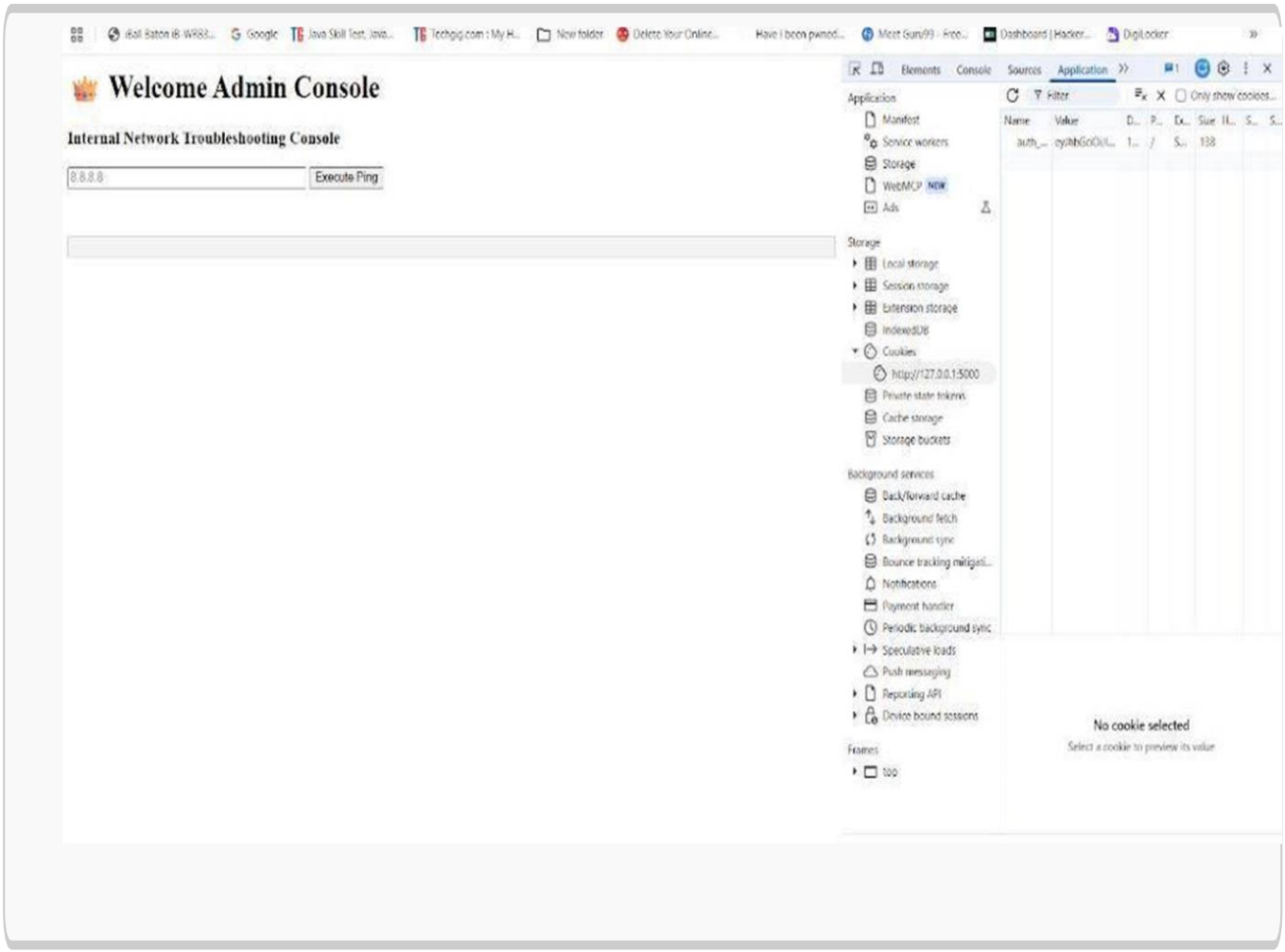
The initial deployment establishes a standard guest connection tier, issuing a client-side session tracker cookie containing authorization claims payload structures. An interrogation of the token properties reveals the utilization of symmetric signature hashes.



This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

Stage 2: Token Forgery & Administrative Privilege Escalation

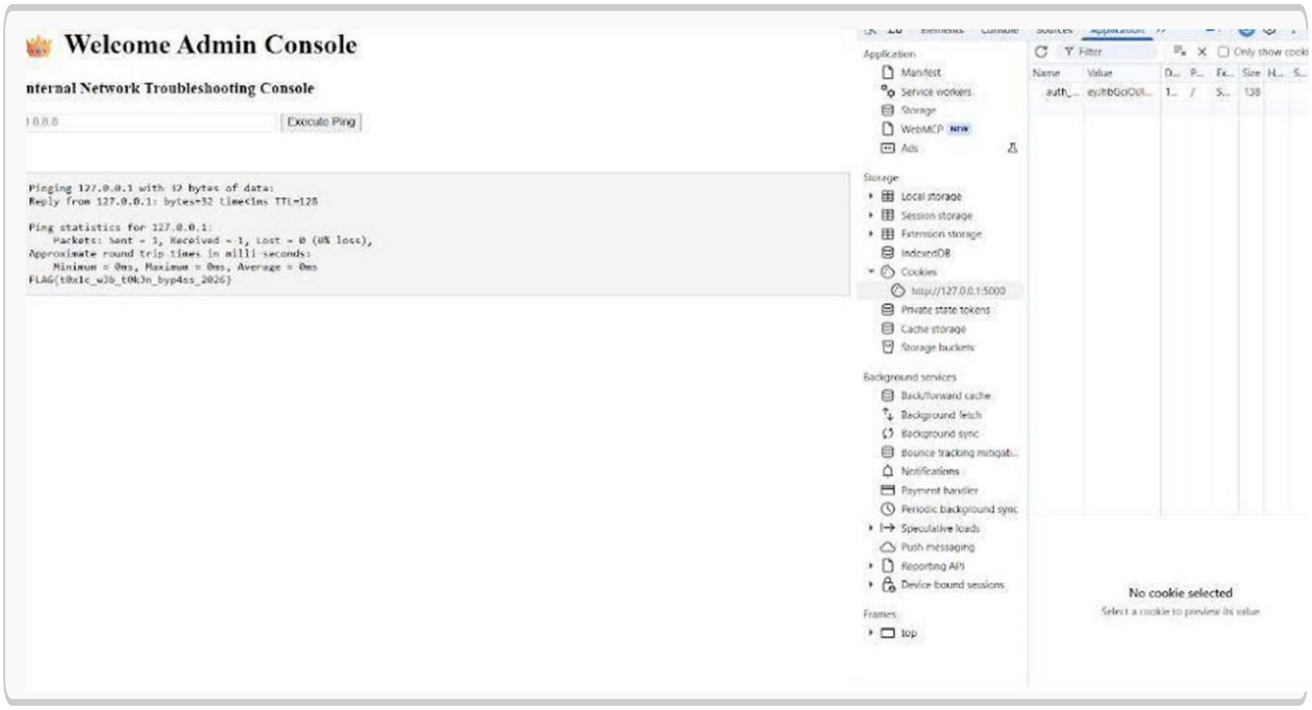
By deploying the cracked signature parameter value within localized manipulation tools, the authorization data claims array is systematically structured to reflect administrative clearance layers. The updated token is then placed directly back into the browser storage framework to force an authorization context shift.



This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

Stage 3: Authenticated Command Chaining & System RCE

Upon successfully authenticating against the hidden /admin/debug dashboard routing vector, the exposed internal platform utility is targeted using operating system character breaker syntax sequences designed to redirect administrative process execution logic.



4. Defensive Remediation & Code Hardening

To effectively secure the web application stack against the demonstrated exploit vectors, developers must apply standard defense-in-depth secure programming principles. Below is the secure, hardened reference implementation deployed within the application codebase:

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.

```
# REMEDIATED SECURE PLATFORM ARCHITECTURE REFERENCE
from flask import Flask, request, make_response, redirect, render_template_string
import jwt, secrets, re, subprocess

app = Flask(__name__)

# DEFENSE 1: Cryptographically Secure Runtime Key Isolation (High Entropy)
JWT_SECRET = secrets.token_hex(32)

@app.route('/admin/debug', methods=['POST'])
def admin_debug():
    # ... Session Token Cryptographic Integrity Checks Enforced ...
    target_ip = request.form.get('ip', '')

    # DEFENSE 2: Strict Character Constraint Sanitization (White-list Pattern
Matching)
    if not re.match(r"^[a-zA-Z0-9.-]+$", target_ip):
        return "Invalid parameters detected.", 400

    # DEFENSE 3: Explicit Parameter Mapping Isolation (Process Shell Execution
Disabled)
    command = ["ping", "-n", "1", target_ip]
    output = subprocess.check_output(command, shell=False, stderr=subprocess.STDOUT,
text=True)
    return render_template_string("<pre>{{out}}</pre>", out=output)
```

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.